```
car_age = gets.to_f    # let's assume it's 9.5
case car_age
when 0...1
  puts "Mmm.. new car smell"
when 1...3
  puts "Nice and new"
when 3...10
  puts "Reliable but slightly dinged"
when 10...30
  puts "Clunker"
else
  puts "Vintage gem"
end
```

produces:

```
Reliable but slightly dinged
```

Note the use of exclusive ranges in the previous example. These are usually the correct choice in `case` statements. If instead we had written the following, we'd get the wrong answer because 9.5 does not fall within any of the ranges, so the `else` clause triggers:

```
car_age = gets.to_f    # let's assume it's 9.5
case car_age
when 0..0
  puts "Mmm.. new car smell"
when 1..2
  puts "Nice and new"
when 3..9
  puts "Reliable but slightly dinged"
when 10..29
  puts "Clunker"
else
  puts "Vintage gem"
end
```

produces:

```
Vintage gem
```

What's Next

In this chapter we covered some of Ruby's most commonly used types—numbers, strings, and ranges. We showed how to create literals and how to use the functionality of these types. Now let's look at one of Ruby's most powerful standard types: regular expressions.